Facade Design Pattern

Structural Design Pattern

Simplifying Complex Systems

SIMPLE INTERFACE.
POWERFUL SYSTEMS.

REDUCE COMPLEXITY.
IMPROVE MAINTAINABILITY.

BETTER DESIGN.
BETTER SOLUTIONS.



DESIGN PATTERNS

Where are we?

We are moving from Creational patterns,
which focus on object creation,
to Structural patterns,
which focus on organizing and simplifying
existing structures.

Creational
- ✓ Singleton
- ✓ Factory Method

Structural
- ▶ Facade ← Today

Behavioral

TODAY MARKS THE TRANSITION

From creating objects to organizing and simplifying systems.



As we move forward, we will see how Structural patterns help us build flexible, maintainable, and easy-to-use systems.

Review Activity

(5 minutes)

Let's review by thinking in terms of **problems and solutions**, not definitions.



Instead of asking,

"What is Singleton?"

VS

Ask students

Which pattern would you use?

Scenario	Pattern
Only one Database Connection	?
Create different Payment objects	?
Hide an entire banking system	?

Think

Focus on the **problem and goal.**

The right pattern is the solution!

Good developers don't just know patterns.
They know **when to use them.**

Creational
- ✓ Singleton
- ✓ Factory Method

Structural
- ▶ Facade

Behavioral

Wedding Planner Activity

Ask

If YOU organize a wedding...

What services are needed?



Students answer

- Venue
- Food
- Photography
- Music
- Decorations
- Transportation
- Guests

Question

Would you call every vendor yourself?

Or

Hire one Wedding Planner?

The planner is the **Facade.**

You interact with one simple interface (the planner) that provides access to many complex services.

Another Example
Restaurant

In a restaurant, many people work together to serve you.

The waiter acts as a **FACADE.**
You place your order, they handle the rest!

Customer never talks directly to everyone.

- Customer
- Waiter
- Kitchen
- Cashier
- Chef
- Dessert Station
- Delivery

The Waiter is the Facade

They provide a simple interface to a complex system behind the scenes.




Kitchen


Cashier


Chef


Dessert Station


Delivery

You enjoy the meal without needing to know how everything works behind the scenes.

Why Do We Need Facade?

Without a Facade, the client must interact with many parts of the system.

High Coupling
Client depends on many classes.

Hard to Maintain
Changes in any subsystem affect the client.

Hard to Test
Difficult to mock or isolate multiple dependencies.

Complex to Use
The client must know how everything works.



With Facade

The Facade provides a **simple interface** to a complex system.

Benefits

Low Coupling
Client depends on only one class.

Easy to Maintain
Changes inside the system do not affect the client.

Easy to Test
Facade can be mocked; subsystems can be tested independently.

Simple to Use
Client interacts with a clean and easy-to-use interface.



Only ONE interaction

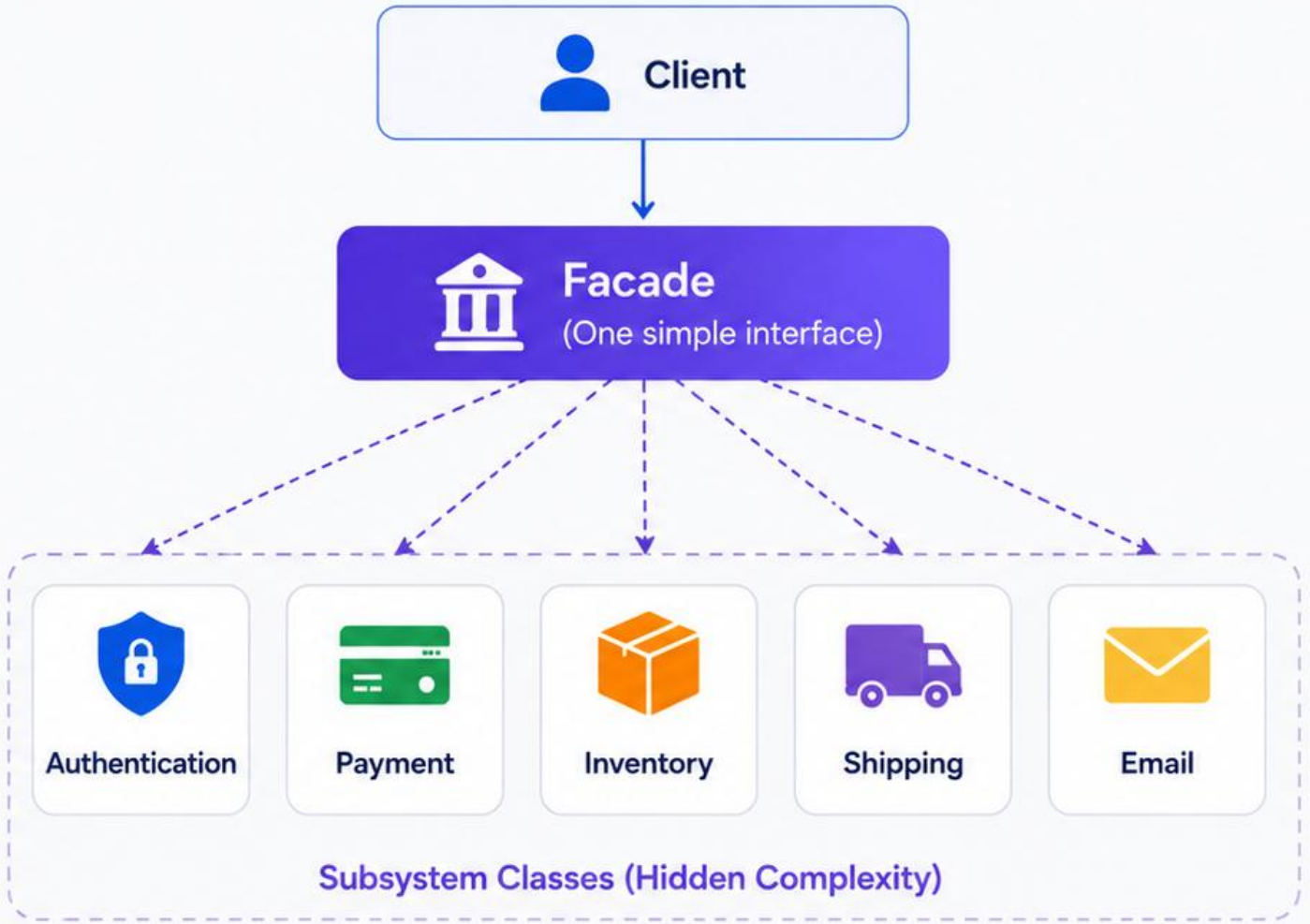
The client interacts with the system through the Facade, hiding all the complexities behind the scenes.

DEFINITION

A Facade Pattern

Provides **one simple interface** that **hides** the **complexity** of multiple subsystem classes.

The client doesn't need to know how the subsystems work.
It just uses the Facade.

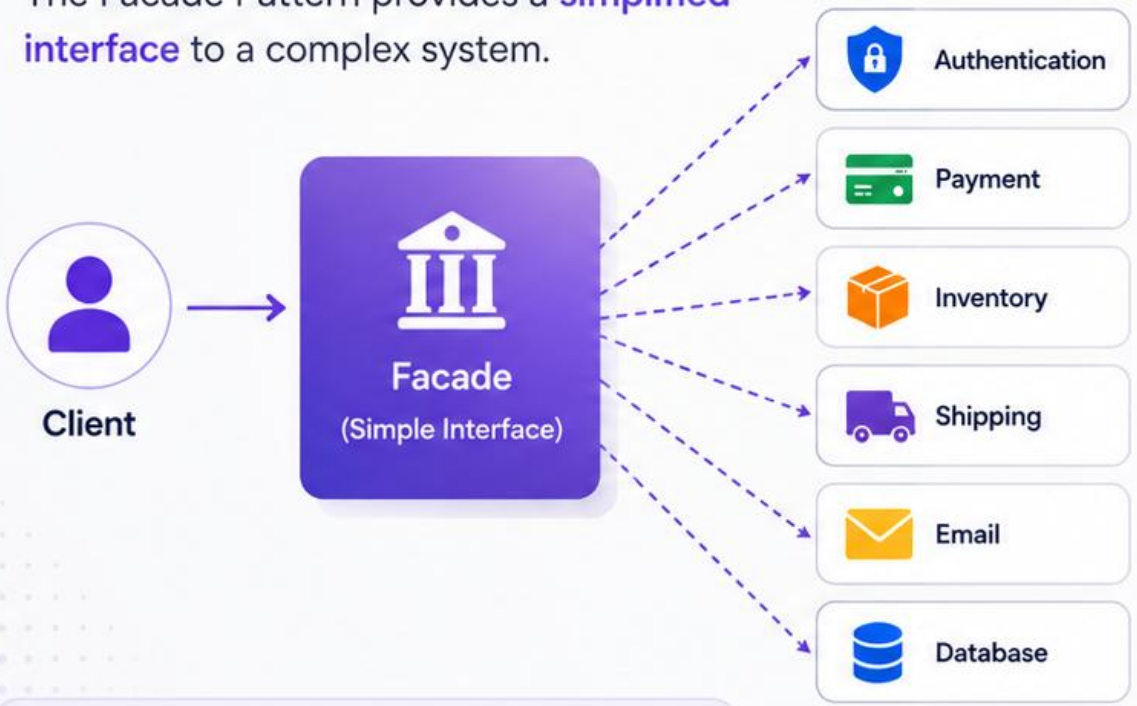


Facade simplifies your world.
One door. Many rooms.



Characteristics of the Facade Pattern

The Facade Pattern provides a **simplified interface** to a complex system.



The Facade hides the complexities of the subsystems and provides a single entry point for the client.

Structural Pattern
Part of the Structural design patterns.

Wrapper Class
Acts as a wrapper over a collection of subsystem classes.

Simplifies APIs
Provides a simple and unified interface to clients.

Reduces Coupling
Decouples the client from complex subsystem classes.

Easy Maintenance
Changes in subsystems have minimal impact on clients.

Coordinates Multiple Subsystems
Manages interactions and workflows between multiple subsystems.

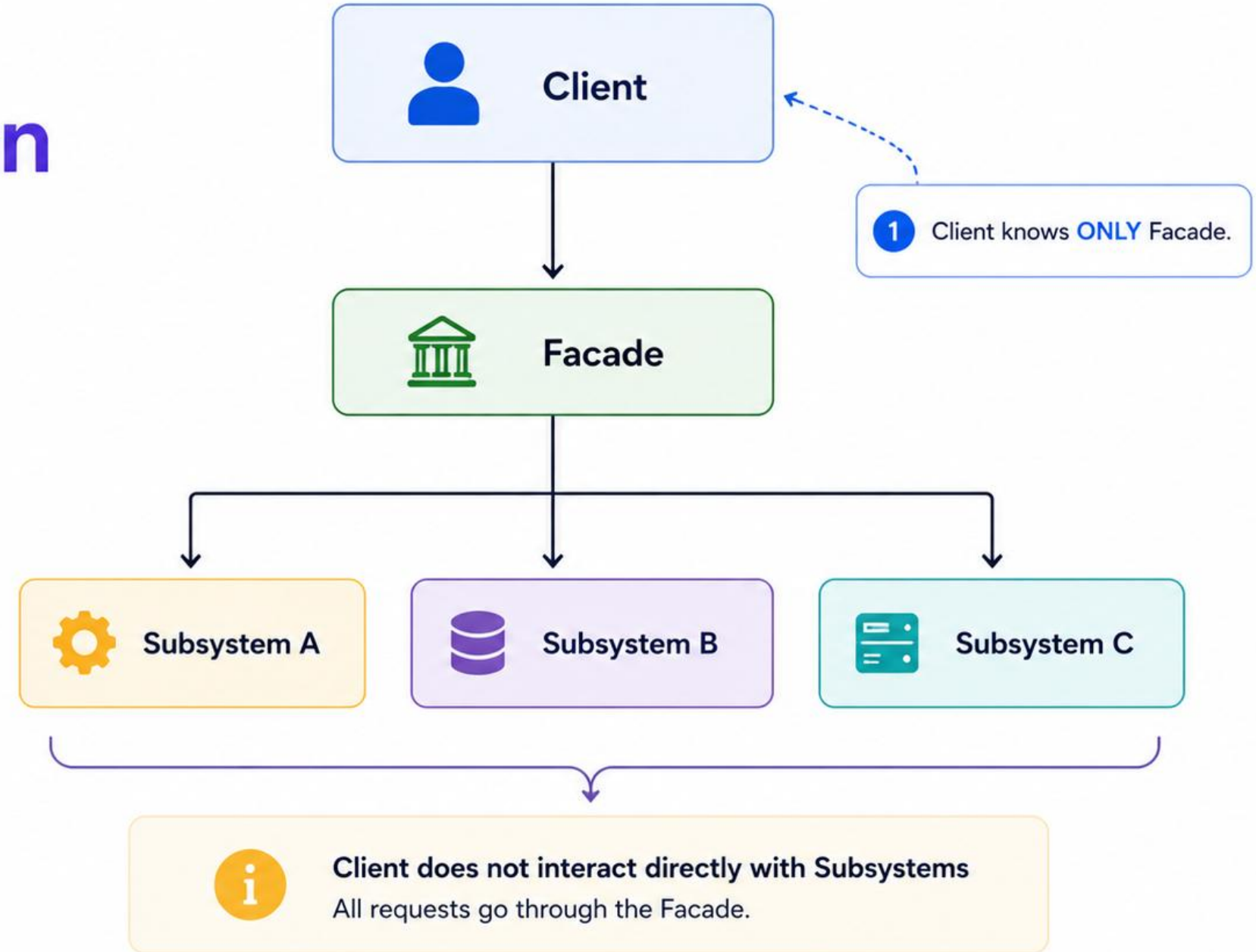
UML Diagram
Facade Pattern

EXPLAIN

Client knows ONLY Facade
The client interacts with the system through the Facade class.

Facade hides complexity
Facade provides a simple interface and delegates the work to subsystem classes.

Subsystems do the work
The subsystems handle the actual processing.



REAL EXAMPLE

Netflix
and the Facade Pattern

Netflix provides a **single simple interface** to the user. All the complex operations happen behind the scenes.





USER

Play Movie

One click. That's all the **user** sees.

BEHIND THE SCENES

All these services work together to play your movie seamlessly.

Authentication	Verifies user identity and permissions.
Subscription Check	Checks if the user has an active subscription.
Recommendation	Suggests content based on user preferences.
CDN Selection	Chooses the best server for fast streaming.
Video Decoder	Prepares and decodes the video stream.
Subtitle Service	Fetches and synchronizes subtitles (if enabled).
Analytics	Tracks viewing data and improves experience.

All hidden. You just enjoy the movie!

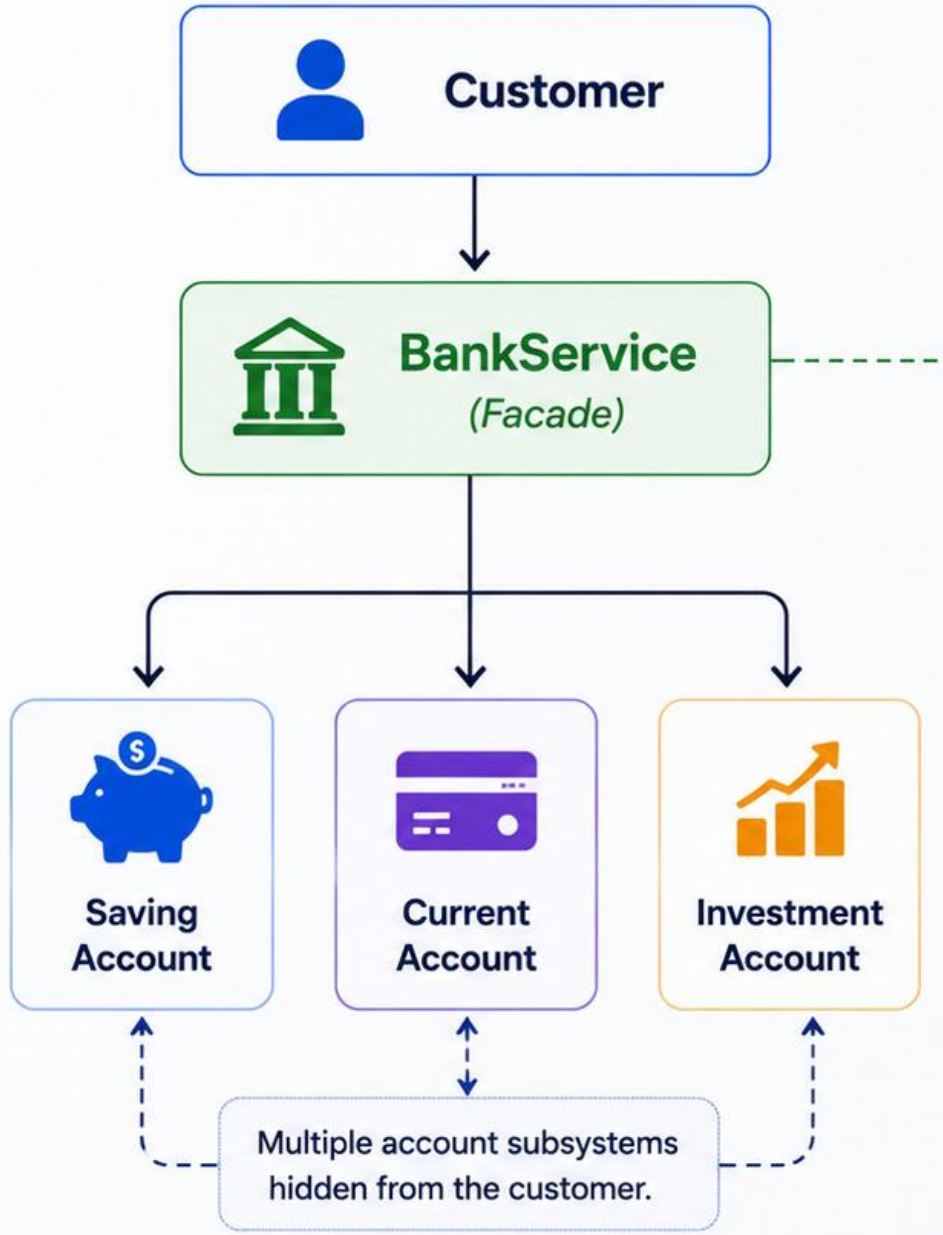
Banking Example

The customer interacts with the bank through a single simple interface called **BankService**. All the account operations are handled behind the scenes.

BankService is the Facade.

- It provides a **simple interface** to the customer.
- It **hides the complexity** of multiple account systems.
- It **coordinates** the work of different account subsystems.

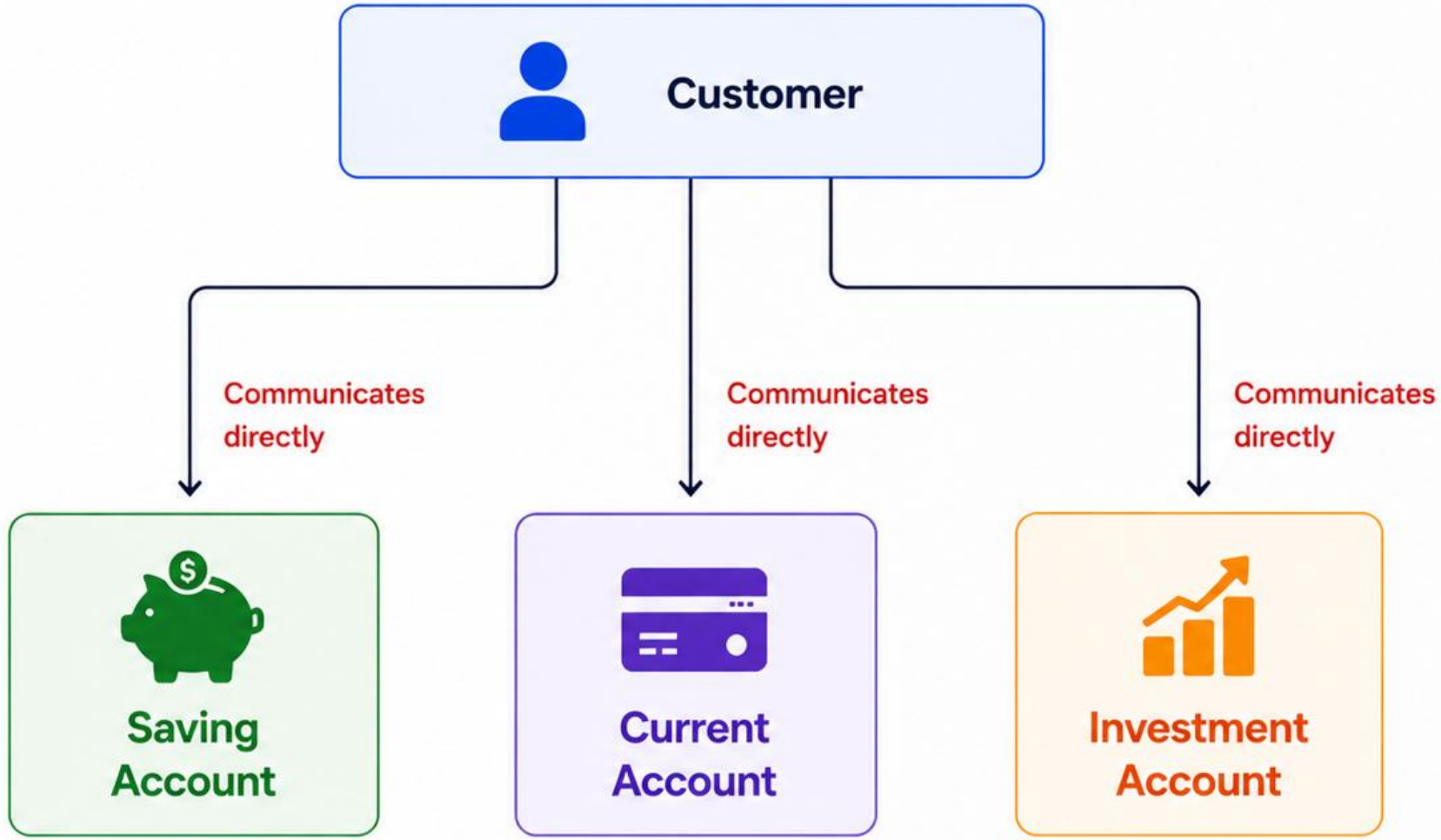
Customer does not need to know how each account works internally. They just use **BankService**.





Before Facade

In this design, the customer must communicate with **every account individually**.



PROBLEMS

- ✕ High coupling between customer and subsystems
- ✕ Difficult to use
- ✕ Hard to maintain
- ✕ Any change in an account affects the customer
- ✕ Code duplication is more likely

RESULT | Customer communicates with every account individually. **Too many dependencies!**

UML NOTE | The diagram above shows **multiple direct associations** between Customer and each subsystem.

After Facade

With Facade, the customer **only** communicates with **BankService**.



Only ONE communication



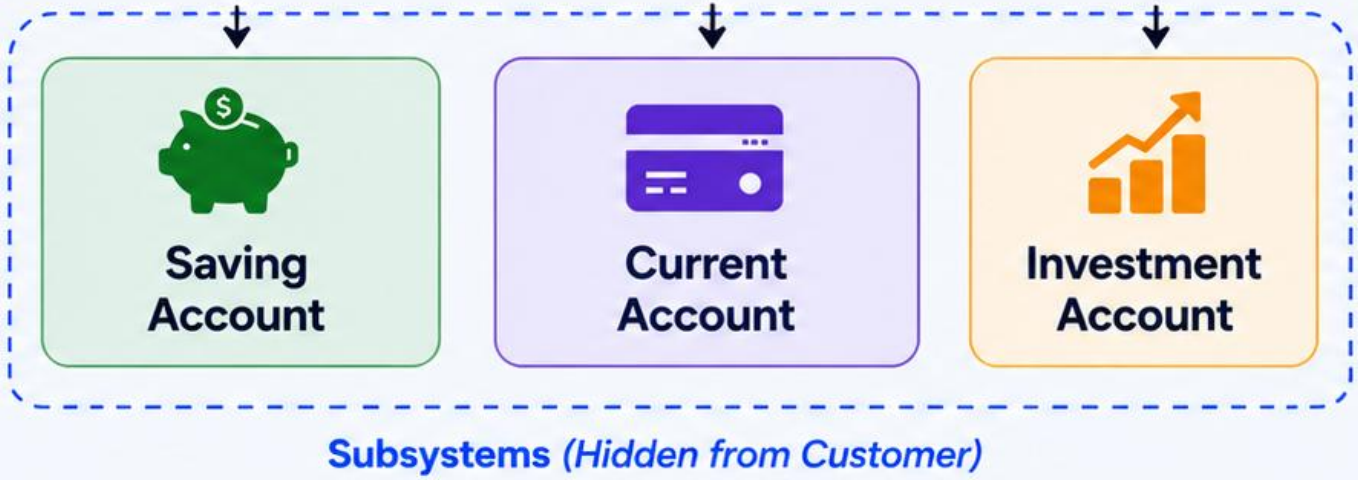
Benefits

- Single point of acess
- Lower coupling
- Easy to use
- Easy to maintain
- Change in subsystem won't affect customer

Key Point
Customer only communicates with **BankService (Facade)**.

Explanation

Customer only communicates with **BankService**. All complexity is hidden behind the Facade.



Subsystems *(Hidden from Customer)*

Result: Customer only communicates with BankService. | **Simpler, Cleaner, Better Design!**

Java Implementation Overview

Facade Pattern – Banking Example

What we are building

A simple banking system using **Facade Pattern**.

Key Idea

Customer does not interact with individual accounts.

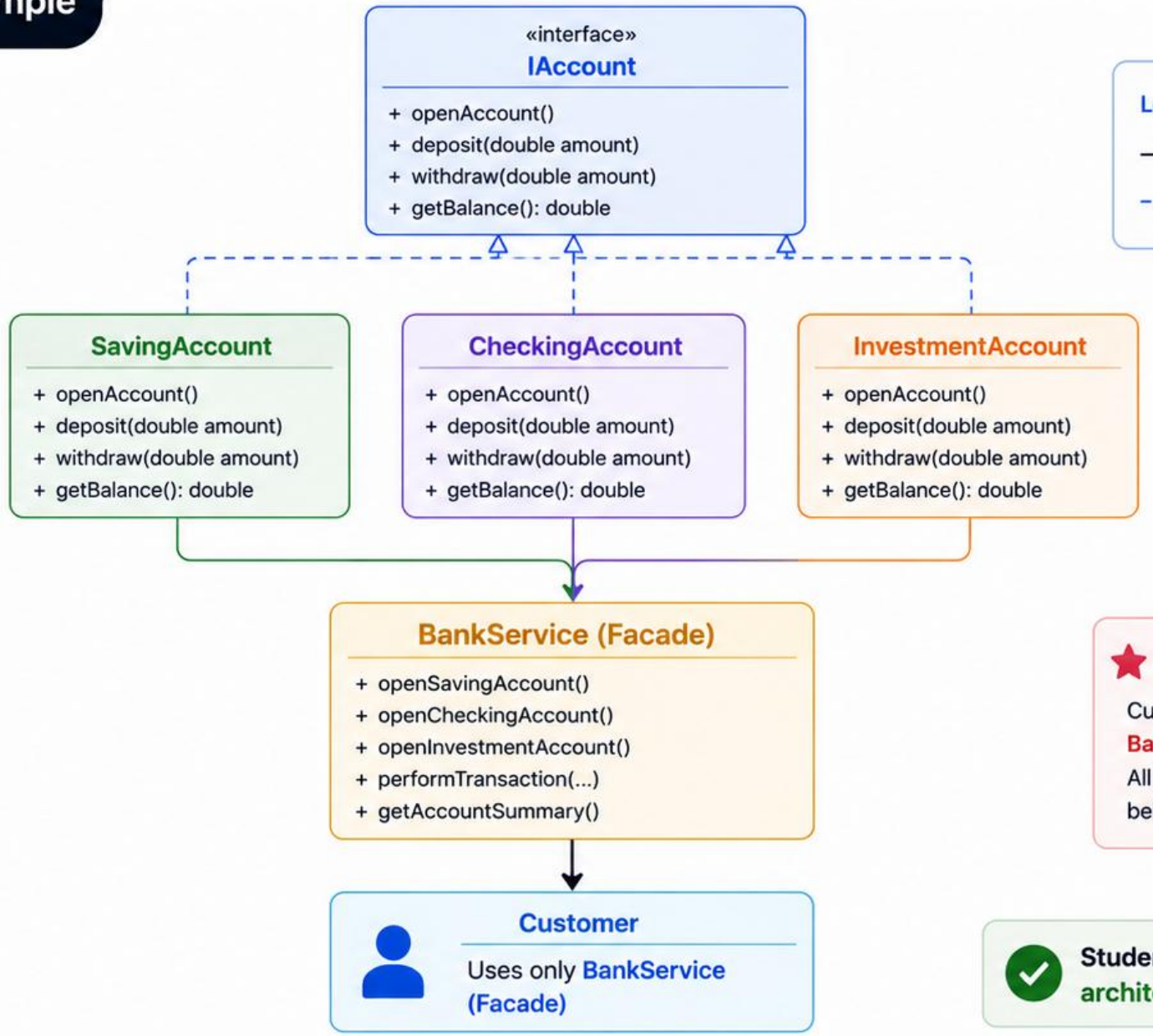
All requests go through **BankService (Facade)**.

Benefits

- ✓ Single entry point for the customer
- ✓ Hides complexity of accounts
- ✓ Loose coupling
- ✓ Easy to extend and maintain

Learning Goal

Students understand the **ARCHITECTURE** before writing code.



Show Interface

(short code only)

What is an Interface?

An interface defines a contract. Classes that implement the interface must provide the method implementations.

Key Points

- Contains abstract methods (no body)
- Methods are public and abstract by default
- Fields are public, static and final by default
- Implemented using the `implements` keyword

1. Interface Definition

```
1  interface Payment {
2     void pay(double amount);
3     String getPaymentStatus();
4  }
5
6  // No method body in interface
```

2. Interface Implementation

```
1  class CreditCardPayment implements Payment {
2     public void pay(double amount) {
3        System.out.println("Paid: " + amount);
4     }
5
6     public String getPaymentStatus() {
7        return "Success";
8     }
9  }
```

Facade – BankService

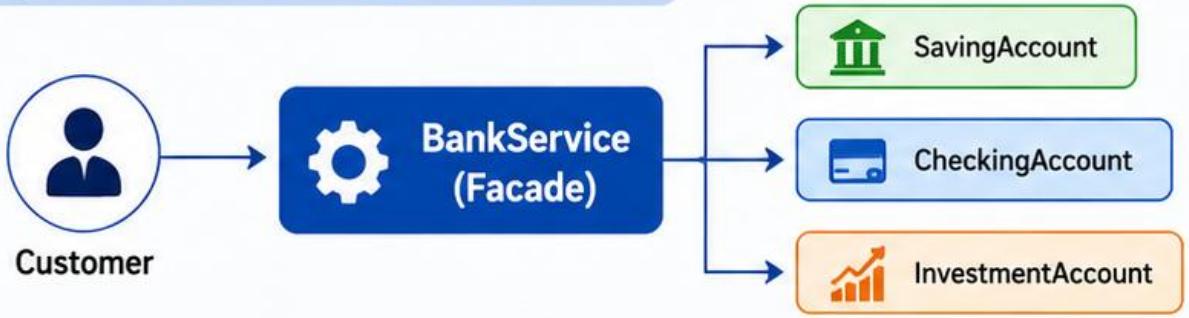
A simplified interface that hides the complexity of the subsystem.



BankService (Facade)

```
1  public interface BankService {
2     void createAccount(Customer c);
3     void transferMoney(String fromAcc, String toAcc, double amount);
4     void withdraw(String accNo, double amount);
5     void deposit(String accNo, double amount);
6  }
```

How BankService (Facade) works



Responsibilities

createAccount()	Creates a new customer account by delegating to the appropriate account subsystem (Saving, Checking, etc.).
transferMoney()	Transfers money from one account to another by coordinating debit and credit operations.
withdraw()	Withdraws the specified amount from the given account after validation.
deposit()	Deposits the specified amount to the given account after validation.

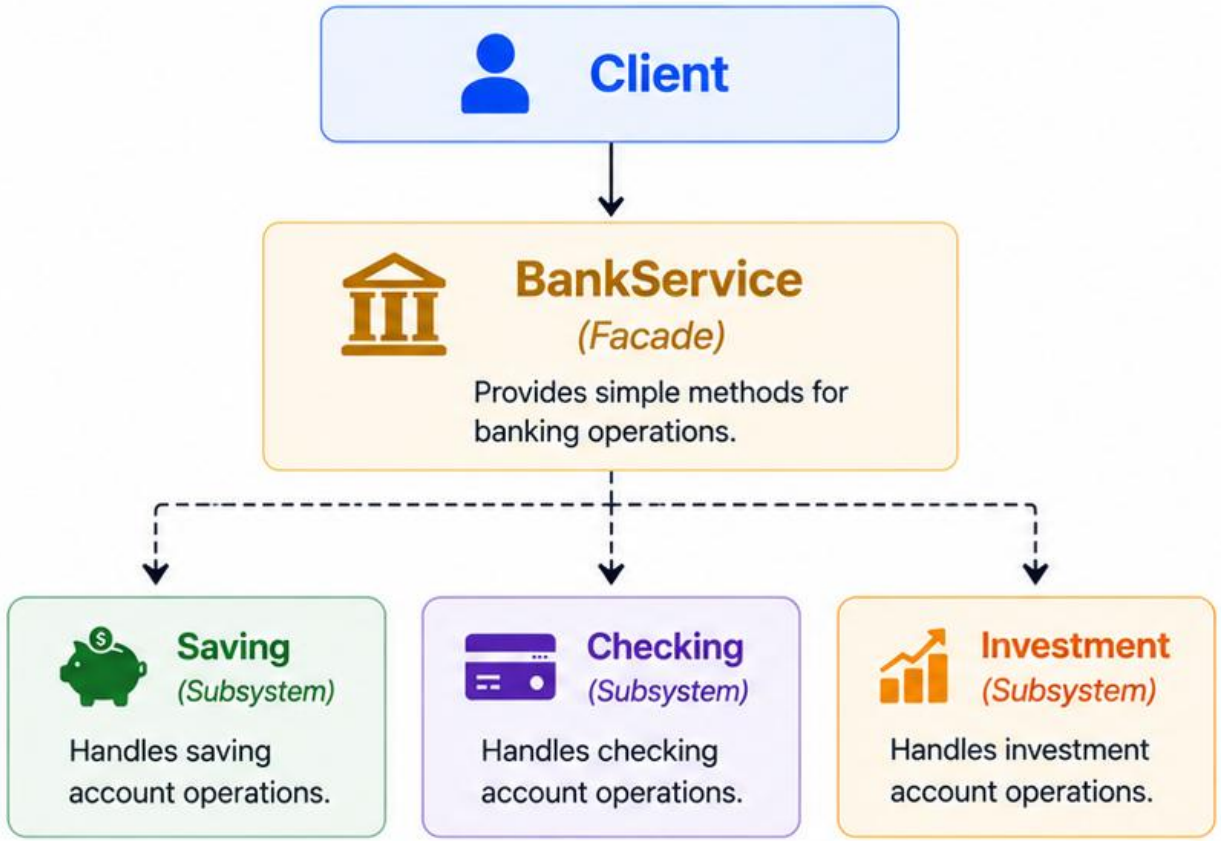
Key Takeaway

BankService provides a single, simple interface for the client (Customer) and hides the complexity of multiple account classes.

Facade Pattern in Action

One simple interaction hides all the complexity.

The Client deals with **ONLY BankService**.



Client Code (Simplified)

Only one object. Only one entry point.

```java
// Client uses only BankService (Facade)
public class Client {
    public static void main(String[] args) {

        // Create Facade object
        BankService bank = new BankService();

        // Perform operations - only through Facade
        bank.transferMoney("John", "Alice", 1000.00);
        bank.deposit("John", 500.00);
        double balance = bank.getBalance("John");
        System.out.println("Balance: " + balance);
    }
}
```

The Client knows and uses **ONLY BankService**.
All account details and complexity are hidden.

Key Takeaway | **ONE object. ONE interface. ZERO worries about internal complexity.**

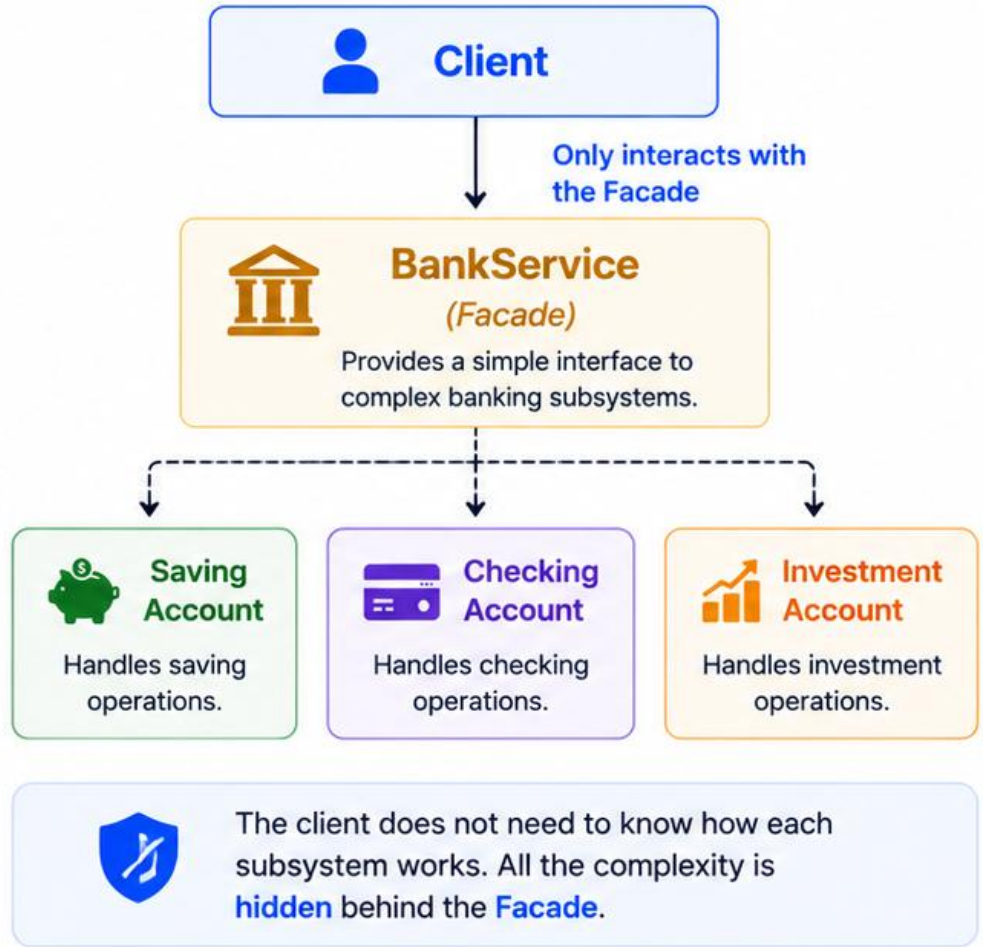
Simpler Code | Loose Coupling | Easy Maintenance

Facade Pattern – Advantages & Limitations

Facade Pattern provides a simplified interface to a complex subsystem of classes.
It improves usability, reduce dependencies, and promotes clean architecture.

Advantages

- **Less Coupling**
  Client depends only on the Facade, not on multiple subsystem classes.
- **Easier to Learn**
  Simplifies the system for new developers.
- **Easier Maintenance**
  Changes in subsystems don't affect the client directly.
- **Cleaner Code**
  Reduces complex interactions and makes code more organized.
- **Reusable**
  Facade can be reused across different clients.
- **Better Readability**
  Client code is short, clear, and easy to understand.



The client does not need to know how each subsystem works. All the complexity is **hidden** behind the **Facade**.

Limitations

- **May become a God Class**
  Facade can grow too large and handle too many responsibilities.
- **Additional Abstraction**
  Adds an extra layer that may not be necessary in simple cases.
- **Not suitable for very small systems**
  Overkill for systems with very few classes or subsystems.
- **Can hide useful subsystem functionality**
  Clients may not access advanced features provided by subsystems.

Key Takeaway: Facade Pattern is excellent for simplifying complex systems, but use it wisely. Keep the Facade focused, minimal, and aligned with the system's needs.

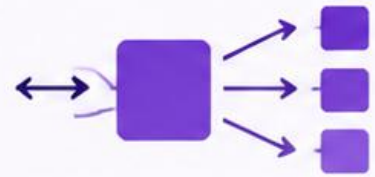
Facade vs Factory Method

Both patterns simplify code, but they solve different problems.
Factory Method (Creational) focuses on **object creation**.
Facade (Structural) focuses on **simplifying complex interactions**.

Factory Method
(Creates Objects)



Facade
(Coordinates Objects)



Aspect	Factory Method (Creational Pattern)	Facade (Structural Pattern)
Purpose	**Creates objects** Provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.	**Coordinates objects** Provides a simplified interface to a complex subsystem by coordinating multiple classes and their interactions.
Category	**Creational Pattern** Focuses on how objects are created.	**Structural Pattern** Focuses on how classes and objects are composed to form larger structures.
What it does	**Returns object** The factory method returns an object (of the appropriate type).	**Executes workflow** The facade executes a sequence of operations involving multiple subsystem objects.
Client focus	Knows about the **factory** and the product interface. Client uses the **factory** to get **objects**.	Knows only about the **facade**. Client uses the **facade** to perform operations.
Goal	Delegate **object creation** to subclasses.	**Hide complexity** of subsystem and provide a simple interface.
Example	**Payment gateway**: CreateCreditCardPayment(), CreatePayPalPayment(), etc.	**Banking system**: transferMoney(), openAccount(), generateStatement() – all handled by BankService.

Remember

Factory Method decides "**which object to create**".
Facade decides "**what to do with multiple objects**".

Key Takeaway

Factory Method is about **creation**.
Facade is about **simplification**.

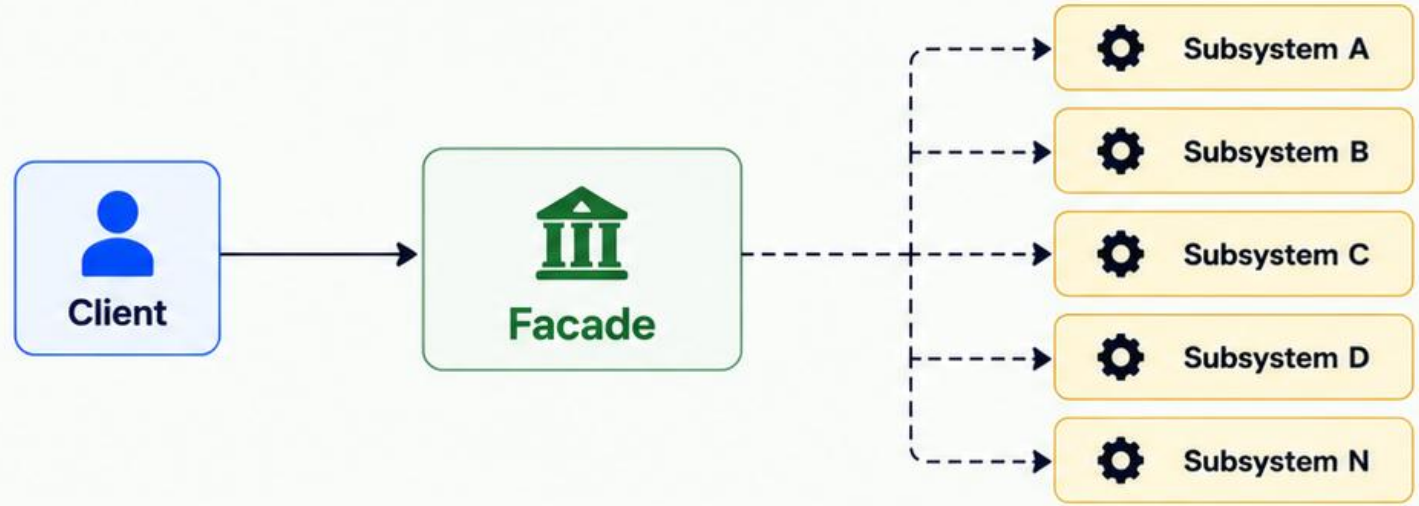
When Should You Use Facade?

Facade provides a **simple interface** to a **complex subsystem**.

Use Facade when:

- **Many subsystem classes exist**
  The system has many classes with complex interdependencies.
- **Complex APIs**
  The API is difficult to understand or use directly.
- **Third-party libraries**
  You want to hide the complexity of external libraries from clients.
- **Banking**
  Simplify operations like accounts, transactions, loans, and more.
- **E-commerce**
  Coordinate services like payment, inventory, shipping, discounts, and notifications.
- **Hospital systems**
  Integrate modules like patient records, billing, appointments, pharmacy, and reports.
- **ERP (Enterprise Resource Planning)**
  Provide a unified interface to multiple modules like HR, Finance, Inventory, Sales, etc.

How Facade Simplifies Interaction



Clients talk to the Facade.
The Facade **coordinates everything** behind the scenes.

Real-World Examples

Banking
Unified access to accounts, loans, transfers, cards, and more.

E-commerce
Handle order, payment, inventory, shipping, discounts, and tracking.

Hospital Systems
Integrate patient records, billing, appointments, pharmacy, etc.

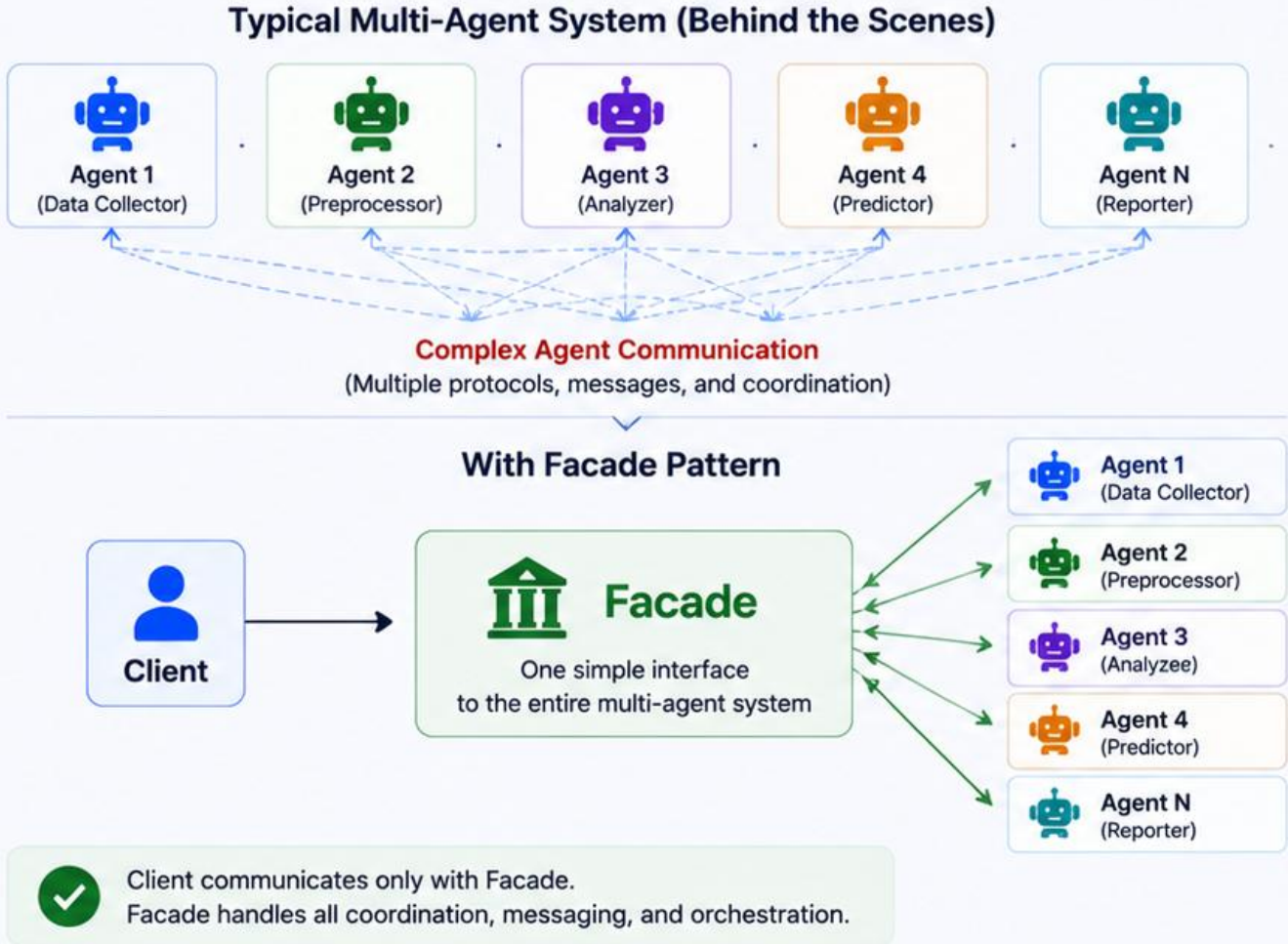
ERP
One interface to HR, Finance, Inventory, Sales, Procurement, etc.

Third-party Libraries
Hide complex APIs behind a simple interface.

Key Takeaway: Use Facade when you want to provide a **simple, clean,** and **unified interface** to a complex set of classes or systems.

Research Connection
Facade Pattern in Multi-Agent Systems

Facade Pattern plays a vital role in Multi-Agent Systems (MAS) architecture.



Why Facade is Commonly Used in Multi-Agent Systems

Hides Agent Communication
The Facade **hides** all message passing, protocols, and interactions between agents from the client.

Provides One Interface
Clients interact with a **single, unified interface** instead of multiple agents or services.

Simplifies Orchestration
The Facade **coordinates** the workflow, task distribution, and results aggregation internally.

Reduces Client Complexity
Clients focus on their goals, not on how agents work together—leading to **simpler, cleaner, and more maintainable** systems.

Key Takeaway:

In Multi-Agent Systems, Facade acts as the **control center**.
It shields clients from complexity and provides a **clean, simple way** to achieve their goals.

FACADE DESIGN PATTERN – SUMMARY

Facade Pattern provides a **simple, unified interface** to a complex subsystem of classes.
It reduces client complexity and improves system usability.

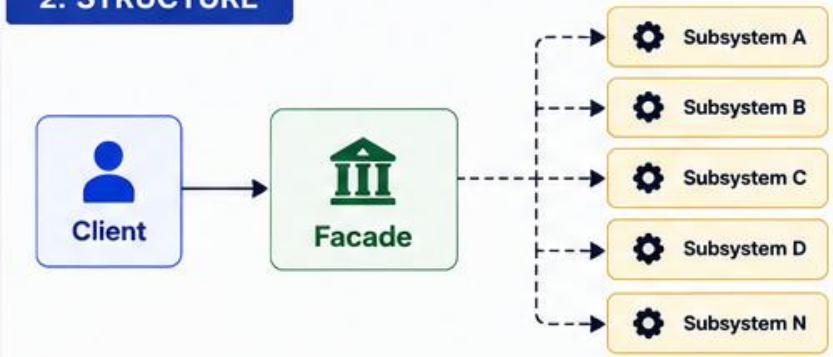
In a Nutshell

Facade acts as a front door that hides the complexity of the system and makes it easy for clients to use.

1. PURPOSE

Provide a **higher-level interface** that makes the subsystem easier to use.

2. STRUCTURE



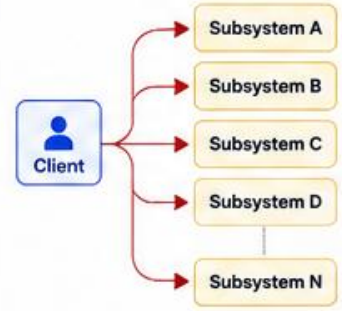
3. HOW IT WORKS

1. Client communicates only with the Facade.
2. Facade coordinates and delegates work to subsystem classes.
3. Subsystems perform the actual work. Client remains unaware of the complexity.

4. BEFORE vs AFTER FACADE

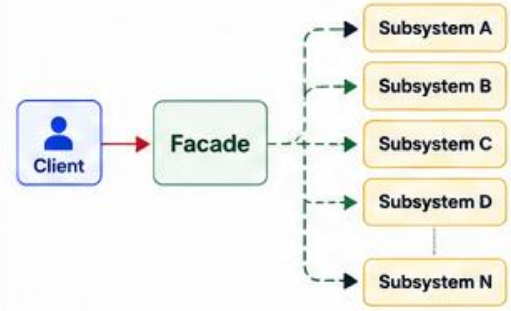
BEFORE FACADE

Client communicates with many subsystem classes.



AFTER FACADE

Client communicates only with Facade.

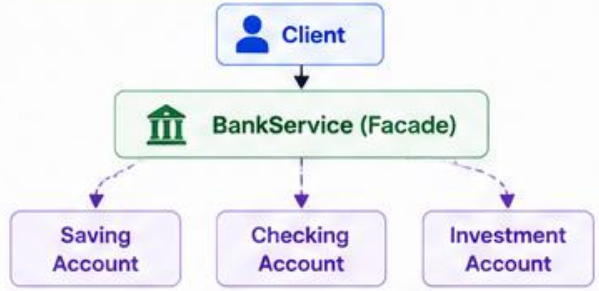


5. WHEN SHOULD YOU USE FACADE?

- Many subsystem classes exist
- Facade APIs that are difficult to understand or use directly
- You are using third-party libraries
- You want to decouple client from subsystem
- Ideal for systems like Banking, E-commerce, Hospital systems, ERP, etc.

Banking
E-commerce
Hospital Systems
ERP

6. EXAMPLE (BANKING)



Client uses BankService for operations like transferMoney(), deposit(), withdraw(), etc.

7. ADVANTAGES

- Less Coupling
- Easier to Learn
- Easier Maintenance
- Cleaner Code
- Reusable
- Better Readability

8. LIMITATIONS

- May become a God Class
- Additional abstraction
- Not suitable for very small systems
- Can hide useful subsystem functionality

9. FACADE vs FACTORY METHOD

Aspect	Factory Method	Facade
Focus	Creates objects	Coordinates objects
Category	Creational Pattern	Structural Pattern

10. RESEARCH CONNECTION (MULTI-AGENT SYSTEMS)

Facade is commonly used in Multi-Agent Systems because:

Hides agent communication
It hides message passing, protocols, and interactions between agents from the client.

Provides one interface
Clients interact with a single, unified interface



11. KEY TAKEAWAY

Facade provides a simple front for a complex world.

It improves usability, reduces dependencies,